

Security+ 2.3 · Buffer Overflows

Mestre CyberSec · Aula 2.3 — Buffer Overflows (3:37) · [Assistir Vídeo no YouTube](#) · Treino Interativo: edusouzamarques.github.io/cybersec

Resumo Executivo da Aula: Escreve MAIS dado do que o buffer aguenta = sobrescreve memória adjacente (pode ser shellcode). Defesa: bounds checking, DEP/ASLR, secure coding, compilar com proteção.

💡 Highlights que a prova CompTIA cobra

- Buffer overflow: escreve além do tamanho do buffer
- DEP: barra execução na heap/stack; ASLR: randomiza endereços

🎯 Dicas de Prova & Cenários Típicos da Banca

- 'application writes beyond allocated memory' = buffer overflow
- 'randomizes memory addresses' = ASLR

Por que este aprofundamento é cobrado na prova?: O vídeo resume os tópicos práticos, mas o exame CompTIA SY0-701 cobra discernimento técnico rigoroso sobre as tecnologias e protocolos específicos desta seção:

• Estouro de Buffer (N/A)

Vulnerabilidade onde uma aplicação escreve dados além do limite de memória alocado de um buffer, podendo sobrescrever memória adjacente e dados como o EIP (ponteiro de instrução).

⚠ **Armadilha CompTIA:** Exam trigger: 'application writes beyond allocated memory' or 'writes more data than the buffer can hold' = Buffer Overflow. Also watch for 'overflows the stack'.

• Estouro de Pilha (N/A)

Tipo de estouro de buffer que visa a região de memória da pilha de chamadas, geralmente sobrescrevendo o endereço de retorno para redirecionar a execução para shellcode do atacante.

⚠ **Armadilha CompTIA:** Exam tip: Stack is associated with function calls, local variables, and return addresses — the classic target for control-flow hijacking attacks.

• Estouro de Heap (N/A)

Estouro de buffer que ocorre na região de memória heap (memória dinâmica), podendo corromper estruturas de dados do programa e levar à execução arbitrária de código.

⚠ **Armadilha CompTIA:** Exam tip: Heap is used for dynamic allocation — distinguish from stack overflow when the scenario mentions dynamically allocated memory or malloc/new.

• Shellcode (N/A)

Payload malicioso de código de máquina injetado pelo atacante, geralmente entregue via estouro de buffer, projetado para abrir um shell ou executar comandos arbitrários no alvo.

⚠ **Armadilha CompTIA:** Exam tip: When the scenario mentions injected code, payloads, or attacker-supplied code following memory corruption = think Shellcode as the attack payload.

• Prevenção de Execução de Dados (DEP)

Recurso de segurança que marca regiões de memória (como heap e pilha) como não executáveis, impedindo que código rode nessas áreas para mitigar a exploração de estouros de buffer.

⚠ **Armadilha CompTIA:** Exam trigger: 'prevents execution in the heap/stack' or 'blocks execution on data regions' = DEP. Don't confuse DEP (memory protection) with DLP (Data Loss Prevention).

• Randomização do Layout do Espaço de Endereços (ASLR)

Técnica de segurança que randomiza os endereços de memória usados por processos do sistema e aplicações, dificultando bastante que ele prevejam e explorem estouros de buffer.

⚠ **Armadilha CompTIA:** Exam trigger: 'randomizes memory addresses' = ASLR. Pairs with DEP as a defense-in-depth strategy against memory-based attacks.

• Verificação de Limites (N/A)

Prática de codificação segura que valida o tamanho da entrada contra o tamanho alocado do buffer antes da escrita, prevenindo condições de estouro de buffer no nível do código-fonte.

⚠ **Armadilha CompTIA:** Exam tip: Bounds checking is the primary preventive control in 'secure coding' answers — choose it when asked how developers mitigate buffer overflows at the source.

• Proteções de Compilador / Stack Canaries (N/A)

Defesas embutidas no compilador (ex.: flag /GS, stack canaries, funções seguras como strncpy) que detectam ou previnem estouros de buffer adicionando verificações de integridade ao código compilado.

⚠ **Armadilha CompTIA:** Exam tip: When the question asks about compiling with protection, enabling compiler flags, or using safe string functions, the answer relates to compiler protections, not ASLR or DEP.

✓ Perguntas de Treino Comentadas (10 Questões de Cenários Reais)

Cenários autênticos alinhados aos objetivos do exame. O gabarito com análise completa dos distratores encontra-se ao final.

QUESTÃO 1 · CENÁRIO DE EXAME

Durante uma revisão de código seguro de um aplicativo em C, um desenvolvedor descobre que a entrada fornecida pelo usuário a partir de um socket de rede é copiada para um array de pilha de 256 bytes usando `strcpy` sem verificar o comprimento da origem. O aplicativo depois trava com entradas longas. A equipe quer remover a vulnerabilidade em sua origem. Qual remediação é MAIS apropriada?

- A) Habilitar Data Execution Prevention em todos os servidores que hospedam o aplicativo.
- B) Substituir a operação de cópia por uma função limitada e rejeitar ou truncar a entrada que exceda o tamanho do destino.
- C) Implantar uma assinatura de sistema de prevenção de intrusões para bloquear strings de entrada longas.
- D) Executar o aplicativo com privilégio mínimo para limitar o impacto caso haja exploração.

QUESTÃO 2 · CENÁRIO DE EXAME

Um testador de penetração envia um payload criado para um serviço vulnerável. O payload sobrescreve um endereço de retorno na pilha e redireciona a execução para um pequeno conjunto de instruções do processador que inicia um shell de comando. Qual termo MELHOR descreve o papel do payload?

- A) Bomba lógica
- B) Shellcode
- C) Rootkit
- D) Keylogger

QUESTÃO 3 · CENÁRIO DE EXAME

Um serviço interno legado é vulnerável a um overflow de buffer de pilha. Durante o teste, o exploit coloca instruções maliciosas na pilha e redireciona a execução para lá, mas as instruções não são executadas e o processo termina com violação de acesso. O SO está configurado com uma proteção que marca a memória da pilha como não executável. Qual proteção é MAIS provavelmente a responsável?

- A) Address Space Layout Randomization
- B) Canário de pilha
- C) Data Execution Prevention
- D) Validação de entrada

QUESTÃO 4 · CENÁRIO DE EXAME

Um analista de segurança revisa um log de falha de aplicativo que mostra `stack smashing detected` imediatamente após uma requisição malformada. O aplicativo foi compilado com uma opção de segurança que coloca um valor aleatório antes do endereço de retorno salvo e o verifica antes de retornar de uma função. Qual proteção está sendo descrita?

- A) Address Space Layout Randomization
- B) Data Execution Prevention
- C) Verificação de limites
- D) Canário de pilha

QUESTÃO 5 · CENÁRIO DE EXAME

Um atacante explora uma vulnerabilidade em um processo de servidor que aloca memória dinamicamente para perfis de usuário. O overflow corrompe metadados adjacentes do heap e depois causa a sobrescrita de um ponteiro de função. Qual tipo de overflow é MAIS provável?

- A) Overflow de pilha
- B) Integer overflow
- C) Overflow de heap
- D) Vulnerabilidade de format string

QUESTÃO 6 · CENÁRIO DE EXAME

Uma empresa quer tornar exploits de buffer overflow menos confiáveis, garantindo que os endereços exatos de memória da pilha, heap e bibliotecas mudem a cada início de processo. Qual controle de segurança deve ser habilitado?

- A) Data Execution Prevention
- B) Address Space Layout Randomization
- C) Canários de pilha
- D) Validação de comprimento de entrada

QUESTÃO 7 · CENÁRIO DE EXAME

Durante um incidente, descobre-se que um processo de servidor web executou um shell de comando após receber um cabeçalho HTTP malicioso. A engenharia reversa mostra que o atacante sobrescreveu o endereço de retorno salvo no quadro de pilha de uma função. Qual vulnerabilidade foi explorada?

- A) Overflow de buffer de pilha
- B) Overflow de buffer de heap
- C) Cross-site scripting
- D) SQL injection

QUESTÃO 8 · CENÁRIO DE EXAME

Uma equipe de desenvolvimento está atualizando um aplicativo em C++ que processa dados de código de barras em arrays de tamanho fixo. O arquiteto de segurança recomenda uma técnica que verifica o tamanho dos dados antes de serem gravados e impede a escrita além do fim do array. Qual técnica está sendo recomendada?

- A) Address Space Layout Randomization
- B) Data Execution Prevention
- C) Verificação de limites
- D) Ofuscação de código

QUESTÃO 9 · CENÁRIO DE EXAME

Um engenheiro de segurança está endurecendo um aplicativo Linux compilado a partir de código C. O objetivo é detectar casos em que uma função grava além de um buffer local de pilha e corrompe o endereço de retorno salvo antes de a função retornar. Qual recurso de endurecimento do compilador é MAIS diretamente projetado para isso?

- A) Executável independente de posição
- B) Stack protector / canário de pilha
- C) Pilha não executável
- D) Address Space Layout Randomization

QUESTÃO 10 · CENÁRIO DE EXAME

Um aplicativo C legado usa `gets()` para ler a entrada do usuário em um buffer de tamanho fixo. Um consultor de segurança diz que essa função é perigosa porque não consegue limitar quantos caracteres são lidos. Qual mudança MELHOR corrige o risco de overflow de buffer?

- A) Compilar o aplicativo com Address Space Layout Randomization.
- B) Substituir `gets()` por uma função que imponha um comprimento máximo de entrada e valide o resultado.
- C) Habilitar Data Execution Prevention no host.
- D) Executar o aplicativo como uma conta de serviço de baixo privilégio.

GABARITO COMENTADO COM ANÁLISE DE DISTRATORES

Questão 1: Alternativa B — Substituir a operação de cópia por uma função limitada e rejeitar ou truncar a entrada que exceda o tamanho do destino.

A causa raiz é uma cópia sem limite para um buffer de pilha de tamanho fixo. Verificação de limites e funções de cópia seguras, ou validação e truncamento da entrada antes da cópia, removem a falha ao garantir que os dados não excedam o destino. DEP, assinaturas de IPS e privilégio mínimo podem reduzir impacto ou exploração, mas não corrigem o código vulnerável.

Questão 2: Alternativa B — Shellcode

Shellcode é o pequeno payload em código de máquina que o atacante injeta e executa após obter controle do ponteiro de instrução. Uma bomba lógica é código acionado por condição, um rootkit oculta acesso e um keylogger registra teclas; nenhum é o código injetado que inicia um shell neste cenário.

Questão 3: Alternativa C — Data Execution Prevention

DEP e NX marcam regiões de memória como pilha e heap como não executáveis. Se o shellcode for colocado lá, a CPU se recusa a executá-lo, geralmente causando violação de acesso. ASLR randomiza endereços, canários de pilha detectam corrupção da pilha e validação de entrada impede entradas malformadas, mas não torna a memória da pilha não executável.

Questão 4: Alternativa D — Canário de pilha

Um canário de pilha é um valor aleatório colocado entre buffers locais e o endereço de retorno salvo. Antes de a função retornar, o canário é verificado; se um overflow de buffer o sobrescrever, o programa detecta a adulteração e termina, gerando mensagens como stack smashing detected. ASLR randomiza endereços, DEP impede execução e verificação de limites impede a escrita.

Questão 5: Alternativa C — Overflow de heap

Overflows de heap ocorrem em memória alocada dinamicamente, o heap. Corromper metadados do heap ou objetos adjacentes para influenciar um ponteiro de função posterior é característico de exploração de heap. Overflows de pilha atingem quadros de pilha; integer overflows e falhas de format string são classes diferentes.

Questão 6: Alternativa B — Address Space Layout Randomization

ASLR randomiza os endereços-base da pilha, heap e bibliotecas a cada início de processo, dificultando prever onde shellcode ou endereços de retorno estão localizados. DEP impede execução de código em memória gravável, canários de pilha detectam corrupção da pilha e validação de entrada impede que entrada maliciosa seja aceita.

Questão 7: Alternativa A — Overflow de buffer de pilha

Sobrescrever um endereço de retorno salvo no quadro de pilha de uma função é o clássico overflow de buffer de pilha. Overflows de heap corrompem memória alocada dinamicamente, XSS afeta navegadores e SQL injection manipula consultas de banco de dados; nenhum corresponde à corrupção de memória descrita.

Questão 8: Alternativa C — Verificação de limites

A verificação de limites verifica se os dados a serem gravados cabem no array ou buffer alocado antes da escrita, impedindo escritas fora dos limites. ASLR e DEP são mitigações de exploração em tempo de execução, e ofuscação de código não impõe segurança de memória.

Questão 9: Alternativa B — Stack protector / canário de pilha

Stack protector e canário de pilha inserem um valor conhecido na pilha e o verificam antes de retornar de uma função, ajudando a detectar corrupção do endereço de retorno salvo. PIE auxilia o ASLR, pilha não executável é um recurso DEP ou NX, e ASLR randomiza localizações de endereço; nenhum verifica diretamente sobrescrita da pilha antes do retorno.

Questão 10: Alternativa B — Substituir gets() por uma função que imponha um comprimento máximo de entrada e valide o resultado.

A causa raiz é o uso de gets(), que não consegue limitar o comprimento da entrada e pode escrever além do buffer. Substituí-lo por uma função de entrada limitada e validar o comprimento corrige a vulnerabilidade. ASLR, DEP e privilégio mínimo são mitigações que podem reduzir exploração ou impacto, mas não removem a leitura perigosa sem limite.

